ByteMart Digital

THE COMPLETE GUIDE

Maven

From Fundamentals to Professional Practice

A complete, example-driven guide to Apache Maven — from the POM, build lifecycle, and dependencies to repositories, plugins, multi-module builds, profiles, and packaging.

30	200+	14	8
Chapters	Code Examples	Visual Diagrams	Modules

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Maven: The Complete Guide

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided “as is”.

Trademarks
Apache Maven is a trademark of the Apache Software Foundation. All other product names, logos, and brands are the property of their respective owners and are used for identification purposes only. An independent educational work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. What Is Maven? 4
2. Installation & Project Structure 7
3. The POM 10
4. Coordinates & the Artifact Model 13
5. Lifecycles, Phases & Goals 16
6. The Default Lifecycle 19
7. Clean & Site Lifecycles 22
8. Running Maven 25
9. Dependency Management Basics 28
10. Dependency Scopes 31
11. Transitive Dependencies & Conflicts 34
12. Managing Versions & BOMs 37
13. Local, Central & Remote Repositories 40
14. Repository Managers 43
15. Publishing Artifacts 45
16. How Plugins Work 48
17. Core Plugins 51
18. Binding Goals to Phases 54
19. Configuring Plugins 57
20. Parent POMs & Inheritance 60
21. Aggregation & the Reactor 63
22. Structuring Multi-Module Projects 66
23. Build Profiles 69
24. Properties & Resource Filtering 72
25. Packaging Types & Assembly 75
26. Executable & Fat JARs 78
27. Testing with Maven 81
28. Maven in CI/CD 84
29. Maven vs Gradle 86
30. Best Practices & Troubleshooting 88

MODULE 1 · MAVEN FOUNDATIONS

CHAPTER 01

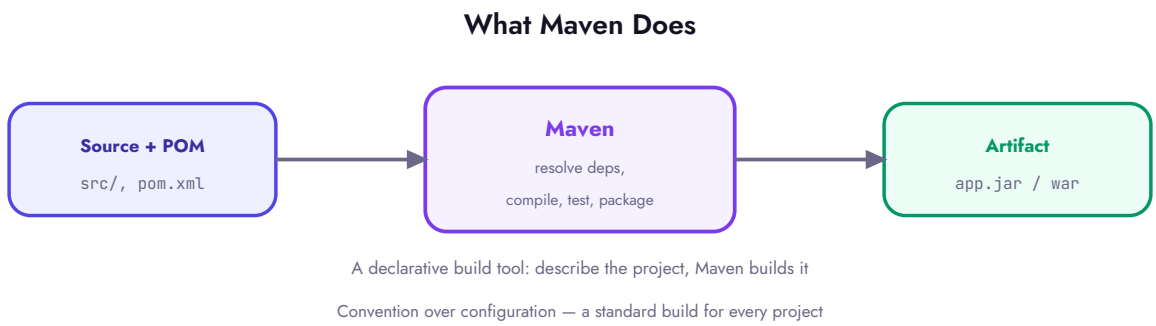
What Is Maven?

A declarative build tool for Java that manages dependencies, compilation, testing, and packaging.

Maven is the most widely used build tool in the Java world. It automates everything between your source code and a finished, distributable artifact — fetching dependencies, compiling, testing, and packaging — all driven by a single declarative file and a standard project structure.

> **Maven**
> A build automation and dependency management tool that builds Java projects from a declarative description (the POM), following convention over configuration.

What Maven does



From source and a POM, Maven resolves dependencies and produces a build artifact.

- **Dependency management** — declare libraries; Maven downloads them and theirs.
- **Standard build** — compile, test, and package with one command.
- **Reproducible** — the same build on every machine and CI server.
- **Extensible** — plugins add capabilities for any build task.

Convention over configuration

Maven's core philosophy: follow its conventions and you write almost no configuration. Put code in the expected folders, and Maven knows how to find, compile, test, and package it — no build script describing every step.

The declarative model

Unlike a script that lists how to build step by step, a Maven build describes the project — its identity, dependencies, and packaging — and Maven derives the steps. You state what the project is; Maven knows how to build it.

```bash
mvn package          # compile, test, and build the artifact
mvn clean install    # rebuild fresh and install locally
```

Maven's place in the ecosystem

- The default build tool for a large share of Java projects.
- Backed by **Maven Central**, the vast public artifact repository.
- Integrated into every IDE and CI system.
- The alternative is Gradle, compared later in this guide.

Describe the project, get a build

Maven turns a declarative project description into a complete, reproducible build — managing dependencies, compilation, testing, and packaging. Follow its conventions and it does the work; that convention-over-configuration model is its defining idea.

© 2026 ByteMart Digital

MODULE 1 · MAVEN FOUNDATIONS

CHAPTER 02

Installation & Project Structure

Setting up Maven and the standard directory layout every project follows.

Getting started with Maven means installing it, verifying the setup, and understanding the **standard directory layout**. That layout is the heart of 'convention over configuration' — because every Maven project is organised the same way, Maven needs almost no configuration to build it.

Installing and verifying

Install Maven (or use the Maven Wrapper, `mvnw`, which pins a version per project) and confirm it and the JDK are visible.

bash

```
mvn -version
# Apache Maven 3.9.x
# Java version: 21, vendor: ...

./mvnw -version   # the wrapper: no global install needed
```

The standard layout



Maven's conventional directory layout — code, resources, tests, and build output.

Directory	Holds
src/main/java	application source code
src/main/resources	config, properties, static files
src/test/java	test source code
src/test/resources	test-only resources
target	build output (generated)

Why the layout matters

Because Maven knows where everything lives, you never tell it where your source or tests are. Deviating from the layout means fighting the tool — so it pays to embrace the convention.

The Maven Wrapper

- `mvnw` / `mvnw.cmd` run a project-pinned Maven version.
- Contributors need no matching Maven install.
- Guarantees everyone builds with the same Maven version.
- Commit it to the repository; use it in CI.

One layout, everywhere

Maven follows a standard directory structure, which is what lets it build with almost no configuration. Learn the layout, use the wrapper for a pinned version, and you get consistent, portable builds across every machine.

© 2026 ByteMart Digital

MODULE 1 · MAVEN FOUNDATIONS

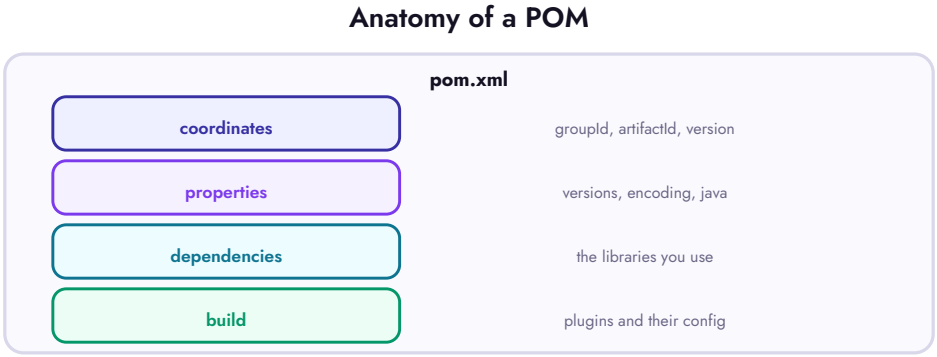
CHAPTER 03

The POM

The Project Object Model — the single XML file that describes and drives every Maven build.

At the centre of every Maven project is the **POM** — the Project Object Model, a file named `pom.xml`. It declares the project's identity, its dependencies, and how it is built. Everything Maven does is driven by this one declarative file.

Anatomy of a POM



A POM declares coordinates, properties, dependencies, and the build configuration.

A minimal POM

xml

```xml
<project>
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.bytemart</groupId>
  <artifactId>order-service</artifactId>
  <version>1.0.0</version>

  <properties>
    <maven.compiler.release>21</maven.compiler.release>
  </properties>

  <dependencies>
    <!-- libraries go here -->
  </dependencies>
</project>
```

The main sections

Section	Declares
coordinates	the project's identity (GAV)
properties	reusable values and settings

Section	Declares
dependencies	the libraries the project uses
build	plugins and build configuration
dependencyManagement	centralised dependency versions

The effective POM

Your POM inherits from a built-in **super POM** (and any parent), so the real, fully-merged configuration is larger than what you write. Inspect it when a setting's origin is unclear.

bash

```
mvn help:effective-pom   # shows the fully-resolved POM
```

One file describes the project

The POM is the declarative heart of a Maven project — identity, dependencies, and build config in one XML file. Maven merges it with inherited POMs into an effective configuration that drives the entire build.

MODULE 1 · MAVEN FOUNDATIONS

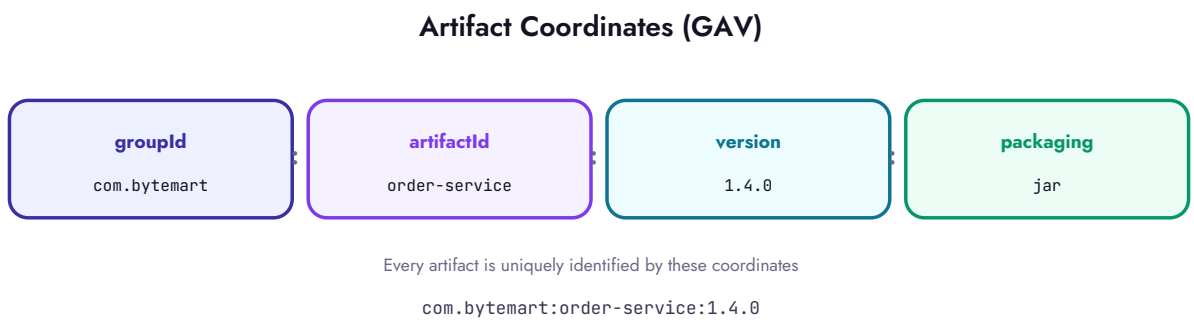
CHAPTER 04

Coordinates & the Artifact Model

How every artifact is uniquely identified by group, artifact, and version.

Maven treats every library and every project as an **artifact** with a unique address. These **coordinates** — group, artifact, version, and packaging — are how Maven names, finds, and resolves everything. Understanding them is understanding how the whole dependency system works.

GAV coordinates



groupId, artifactId, version, and packaging uniquely identify every artifact.

Coordinate	Meaning
groupId	the organisation / namespace
artifactId	the project's name
version	the specific release
packaging	the artifact type (jar, war, pom)

Declaring a dependency by coordinates

xml

```
<dependency>
  <groupId>org.apache.commons</groupId>
  <artifactId>commons-lang3</artifactId>
  <version>3.14.0</version>
</dependency>
<!-- shorthand: org.apache.commons:commons-lang3:3.14.0 -->
```

Versioning and SNAPSHOTs

A version ending in `-SNAPSHOT` marks an in-development, mutable release that Maven re-checks for updates. A version without it is a **release** — immutable, and never expected to change.

- **Release** versions (`1.4.0`) are immutable — built once, never changed.

- **SNAPSHOT** versions (`1.5.0-SNAPSHOT`) are volatile during development.
- Semantic versioning (`MAJOR.MINOR.PATCH`) is the common convention.
- Never depend on a SNAPSHOT in a released build — it is not reproducible.

WATCH OUT

A released artifact must be reproducible, so its dependencies must be fixed release versions. Depending on a `-SNAPSHOT` means your build could produce different results tomorrow — avoid it outside active development.

Every artifact has a unique address

GAV coordinates — groupId, artifactId, version — uniquely identify every artifact Maven handles. Releases are immutable and reproducible; SNAPSHOTs are for development. This naming model is the foundation of dependency resolution.

© 2026 ByteMart Digital

MODULE 2 · THE BUILD LIFECYCLE

CHAPTER 05

Lifecycles, Phases & Goals

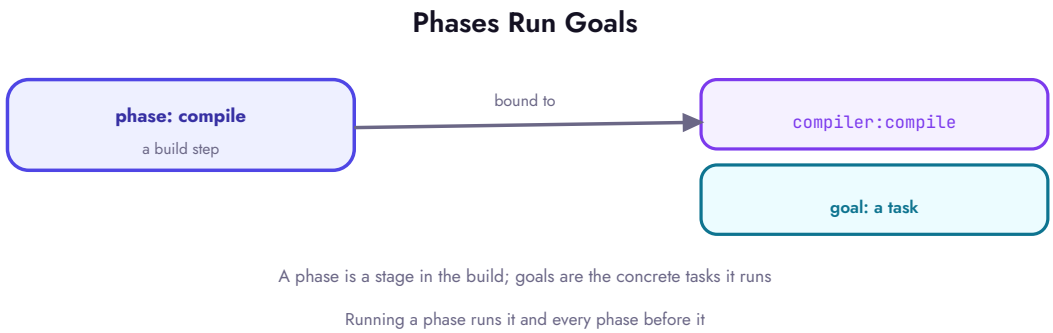
Maven's three-level build model — how lifecycles, phases, and goals fit together.

Every Maven build runs through a well-defined model of **lifecycles**, **phases**, and **goals**. Grasping these three levels — and how running a phase triggers everything before it — demystifies what actually happens when you type `mvn package`.

The three levels

Concept	Is
Lifecycle	an ordered sequence of phases
Phase	a single stage in the build
Goal	a specific task a plugin performs

Phases run goals



A phase is a build stage; goals are the concrete tasks bound to it.

A phase on its own does nothing — it is a slot. **Goals**, provided by plugins, are bound to phases and do the real work. Running a phase runs all the goals bound to it.

Running a phase runs the ones before it

Phases are sequential. Invoking a phase executes it and every phase before it in the lifecycle. This is why `mvn package` also compiles and tests — those phases come first.

```bash
mvn test      # runs validate -> compile -> test
mvn package   # ...all of the above, then package
mvn install   # ...then install into the local repository
```

The three built-in lifecycles

- **default** — builds and deploys the project (the main one).
- **clean** — removes previous build output.
- **site** — generates project documentation.
- Each is independent, with its own ordered phases.

Lifecycle, phase, goal

Maven builds run through lifecycles made of ordered phases, and plugins bind goals to those phases to do the work. Invoking a phase runs it and all preceding phases — the key mental model behind every Maven command.

© 2026 ByteMart Digital

MODULE 2 · THE BUILD LIFECYCLE

CHAPTER 06

The Default Lifecycle

The main build sequence — from validate through compile, test, package, install, and deploy.

The **default lifecycle** is the one you use constantly. It carries a project from raw source to a deployed artifact through an ordered series of phases. Knowing what each phase does — and the order — tells you exactly what any build command will run.

The key phases in order

The Default Build Lifecycle



Phases run in order; invoking one runs all the phases before it too

mvn package runs validate, compile, test, then package

validate, compile, test, package, verify, install, deploy — run in order.

Phase	Does
validate	check the project is correct
compile	compile main source
test	run unit tests
package	build the jar / war
verify	run integration checks
install	copy artifact to local repo
deploy	publish to a remote repo

The common commands

bash

```
mvn compile    # compile main sources
mvn test       # + run unit tests
mvn package    # + produce the jar in target/
mvn install    # + install into ~/.m2 for other local projects
mvn deploy     # + publish to a shared remote repository
```

install vs deploy

- **install** — puts the artifact in your **local** repository, for other projects on your machine.
- **deploy** — publishes it to a **remote** repository, for the whole team.
- Both first run every earlier phase (compile, test, package).
- `mvn clean install` is the everyday 'full rebuild' command.

TIP

`mvn clean install` gives you a reliable, from-scratch build: `clean` deletes old output so nothing stale lingers, and `install` runs the whole default lifecycle up to installing the artifact locally.

Source to artifact, in phases

The default lifecycle runs validate, compile, test, package, verify, install, and deploy in order. Each command runs its phase and all before it — `install` shares locally, `deploy` shares with the team.

© 2026 ByteMart Digital

MODULE 2 · THE BUILD LIFECYCLE

CHAPTER 07

Clean & Site Lifecycles

The two lifecycles beyond the default — cleaning build output and generating documentation.

Besides the default lifecycle, Maven has two more: **clean**, which removes build output, and **site**, which generates project documentation. They run independently of the default lifecycle, and clean in particular is part of nearly every build command you type.

The clean lifecycle

The `clean` lifecycle deletes the `target` directory, removing all generated output so the next build starts fresh. It is separate from the default lifecycle, which is why you combine them.

```bash
mvn clean            # delete target/
mvn clean package    # clean, then run the default lifecycle to package
mvn clean install    # the everyday full-rebuild command
```

Why clean matters

- Removes **stale output** from previous builds.
- Guarantees the artifact reflects the current source.
- Essential in CI, where a clean build is expected.
- Not needed for every local build — incremental builds are faster.

The site lifecycle

The `site` lifecycle generates a documentation website for the project — reports on dependencies, plugins, test results, and more — from POM information and plugins.

```bash
mvn site   # generate the project documentation site in target/site
```

When to use each

- **clean** — constantly, usually combined with a default-lifecycle command.
- **site** — occasionally, to produce project reports and docs.
- The **default** lifecycle remains the one you run most.
- All three are independent — you invoke the phases you need.

> **Clean often, site occasionally**
> The clean lifecycle deletes build output for a fresh start — hence the ubiquitous `clean install` — while the site lifecycle generates project documentation. Both run independently of the default lifecycle you use day to day.

© 2026 ByteMart Digital

© 2026 ByteMart Digital

MODULE 2 · THE BUILD LIFECYCLE

CHAPTER 08

Running Maven

The command-line in practice — essential commands, flags, and how to work efficiently.

Day-to-day Maven is a handful of commands and flags. Knowing the essentials — how to skip tests when appropriate, run offline, build a single module, and get diagnostic output — makes you far more productive at the command line.

Everyday commands

bash

```
mvn clean install       # full rebuild, install locally
mvn test                # compile and run tests
mvn package             # build the artifact
mvn dependency:tree     # show the dependency graph
```

Useful flags

Flag	Effect
-DskipTests	compile tests but don't run them
-o	work offline (no downloads)
-pl	build only one module
-am	also build required modules
-X	debug output for troubleshooting

Skipping tests — carefully

bash

```
mvn install -DskipTests          # skip running tests (still compiles them)
mvn install -Dmaven.test.skip=true   # skip compiling AND running (avoid)
```

WATCH OUT

Skipping tests should be a rare, deliberate choice — for a quick local iteration, not a habit. Never skip tests in CI or before deploying; the whole point of the build is to catch problems, and skipped tests catch nothing.

Passing properties and profiles

```bash
mvn package -P prod              # activate the 'prod' profile
mvn package -Drevision=1.4.2     # set a property on the command line
```

A few commands, used well

Most Maven work is `clean install`, `test`, and `dependency:tree`, plus flags to target modules, work offline, or debug. Skip tests only for quick local iteration — never in CI or before a release.

MODULE 3 · DEPENDENCIES

CHAPTER 09

Dependency Management Basics

Declaring the libraries a project needs and letting Maven fetch them automatically.

Dependency management is Maven's most valued feature. Instead of hunting down JAR files and adding them by hand, you declare the libraries you need by their coordinates, and Maven downloads them — and everything they depend on — automatically.

Declaring a dependency

List each library in the `<dependencies>` section by its group, artifact, and version. Maven fetches it and adds it to the right classpath.

```xml
<dependencies>
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.17.0</version>
  </dependency>
</dependencies>
```

What Maven does with it

- Downloads the JAR from a repository (once, then caches it).
- Pulls in the library's own dependencies automatically.
- Adds everything to the compile, test, or runtime classpath.
- Resolves version conflicts across the whole dependency graph.

Inspecting dependencies

Two commands are invaluable: the dependency **tree** shows what you actually pull in, and the **analyze** goal flags declared-but-unused and used-but-undeclared dependencies.

```bash
mvn dependency:tree      # the full graph of what you depend on
mvn dependency:analyze   # find unused / undeclared dependencies
```

Keep dependencies clean

- Declare only what you **directly** use.
- Do not declare transitive dependencies you get for free — unless you rely on them directly.
- Remove unused dependencies to shrink and clarify the build.
- Review the tree when a build behaves unexpectedly.

© 2026 ByteMart Digital

Declare, and Maven fetches

You declare the libraries you need by coordinates, and Maven downloads them and their dependencies, wiring them onto the right classpath. Keep your declarations to what you directly use, and inspect the tree when things surprise you.

© 2026 ByteMart Digital

MODULE 3 · DEPENDENCIES

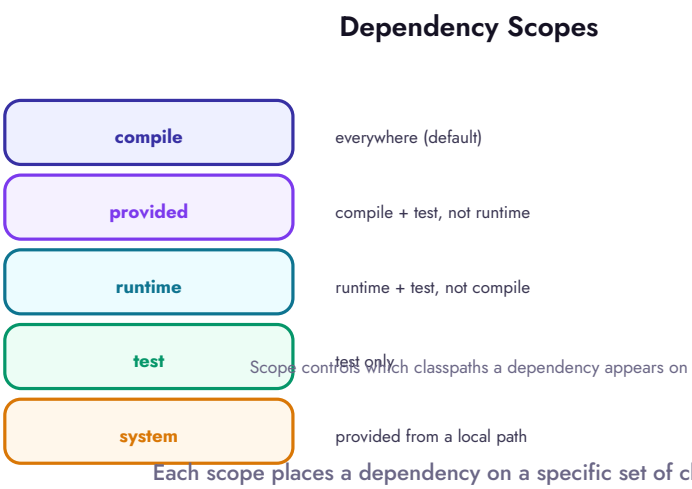
CHAPTER 10

Dependency Scopes

Controlling where a dependency is available — compile, runtime, test, and beyond.

Not every dependency belongs everywhere. A test library is not needed at runtime; a servlet API is provided by the server, not your JAR. **Scope** controls which classpaths a dependency appears on — keeping your build correct and your artifact lean.

The scopes



Scope	Available at
compile	everywhere (the default)
provided	compile & test, not packaged
runtime	runtime & test, not compile
test	test only
system	provided from a local path

Common uses

```xml
<dependency>            <!-- test only -->
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>

<dependency>            <!-- provided by the server at runtime -->
  <groupId>jakarta.servlet</groupId>
  <artifactId>jakarta.servlet-api</artifactId>
  <version>6.0.0</version>
  <scope>provided</scope>
</dependency>
```

Choosing the right scope

- **compile** — libraries your code uses directly (the default).
- **provided** — supplied by the runtime environment (server, container).
- **runtime** — needed to run but not to compile (e.g. a JDBC driver).
- **test** — testing frameworks and helpers, kept out of the artifact.

TIP

Getting scope right keeps your packaged artifact small and correct: test and provided dependencies are excluded from it, so a JDBC driver at `runtime` ships while JUnit at `test` does not.

Scope controls where it applies

A dependency's scope decides which classpaths it appears on — compile, runtime, test, or provided. Choosing correctly keeps builds accurate and artifacts lean, shipping only what the running application actually needs.

© 2026 ByteMart Digital

MODULE 3 · DEPENDENCIES

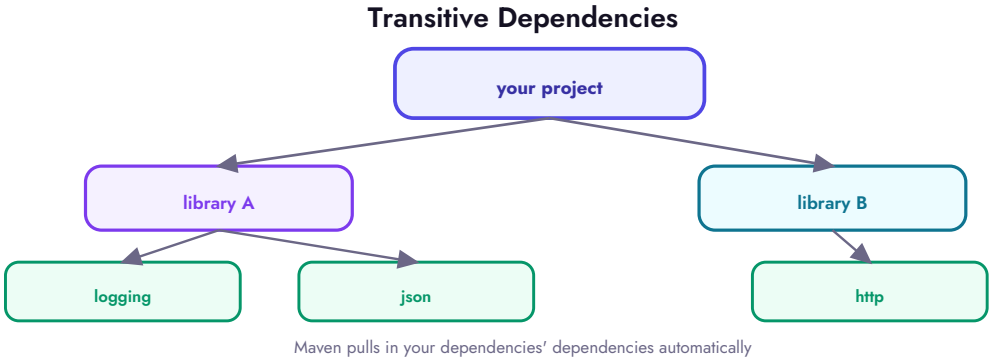
CHAPTER 11

Transitive Dependencies & Conflicts

How Maven pulls in dependencies-of-dependencies, and how it resolves version clashes.

When you depend on a library, you also depend on its dependencies — and theirs. These **transitive dependencies** are pulled in automatically, which is convenient until two paths demand different versions of the same library. Maven has clear rules for resolving these conflicts.

Transitive dependencies



Maven automatically pulls in your dependencies' dependencies, recursively.

You declare a few direct dependencies; Maven resolves the full graph, often dozens of libraries deep. This is powerful, but it means version conflicts are inevitable in large projects.

Conflict resolution: nearest wins

When the graph contains two versions of the same artifact, Maven uses **dependency mediation**: the version **nearest the root** of the tree wins. If two are equally near, the one declared first wins.



When versions clash, Maven picks the one nearest the project in the dependency tree.

Diagnosing conflicts

bash

```
mvn dependency:tree -Dverbose   # shows omitted versions and why
mvn dependency:tree -Dincludes=com.google.guava:guava
```

Controlling the outcome

- **Declare it directly** — a direct dependency is nearest, so it wins.
- **Exclude** an unwanted transitive version (next chapter).
- **dependencyManagement** — pin one version across the whole build.
- Use `-Dverbose` to see exactly which versions were dropped.

WATCH OUT

Version conflicts can cause subtle runtime failures — a `NoSuchMethodError` when the resolved version lacks a method a library expects. When you see one, check the dependency tree for a mediated version mismatch.

Transitive, with nearest-wins

Maven pulls in transitive dependencies automatically and resolves version clashes by choosing the one nearest the root. Understand this to diagnose conflicts — and control the outcome by declaring or managing the version you want.

© 2026 ByteMart Digital

MODULE 3 · DEPENDENCIES

CHAPTER 12

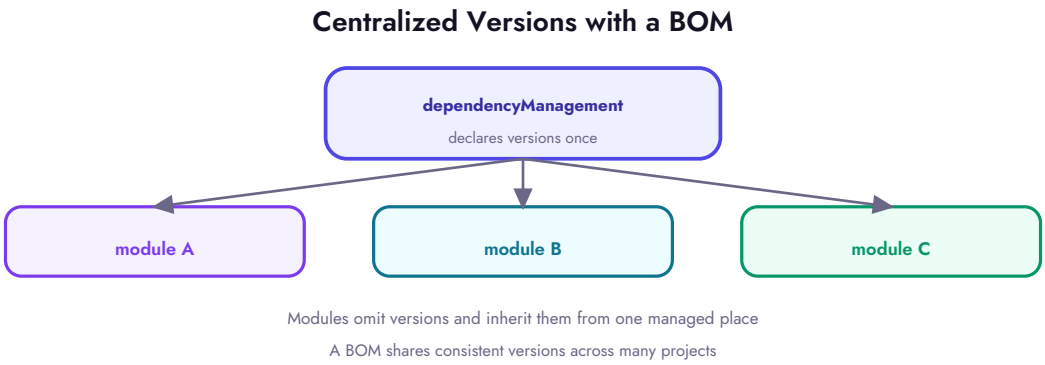
Managing Versions & BOMs

Centralising dependency versions with dependencyManagement, exclusions, and Bills of Materials.

In a real project, versions sprawl — the same library requested at different versions across modules and transitive paths. `dependencyManagement` centralises version decisions, **exclusions** remove unwanted transitives, and a **BOM** shares a consistent set of versions across projects.

dependencyManagement

A `<dependencyManagement>` section declares versions without adding dependencies. Modules and dependency declarations then omit the version and inherit it — one place to control every version.



dependencyManagement declares versions once; modules inherit them without repeating.

```xml
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>com.fasterxml.jackson.core</groupId>
      <artifactId>jackson-databind</artifactId>
      <version>2.17.0</version>
    </dependency>
  </dependencies>
</dependencyManagement>
<!-- elsewhere, just declare it without a version -->
```

Exclusions

To drop an unwanted transitive dependency — a conflicting or vulnerable one — **exclude** it from the dependency that brings it in.

```xml
<dependency>
  <groupId>com.example</groupId>
  <artifactId>big-lib</artifactId>
  <version>4.0.0</version>
  <exclusions>
    <exclusion>
      <groupId>commons-logging</groupId>
      <artifactId>commons-logging</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

Bills of Materials (BOMs)

A **BOM** is a POM that only declares managed versions for a family of libraries. Importing it aligns every related dependency to a tested, compatible set — how Spring Boot keeps dozens of libraries in sync.

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-dependencies</artifactId>
  <version>3.3.0</version>
  <type>pom</type>
  <scope>import</scope>
</dependency>
```

Version decisions in one place

`dependencyManagement` centralises versions, exclusions remove unwanted transitives, and a BOM shares a consistent, tested version set across projects. Together they turn dependency sprawl into a single, controllable source of truth.

MODULE 4 · REPOSITORIES

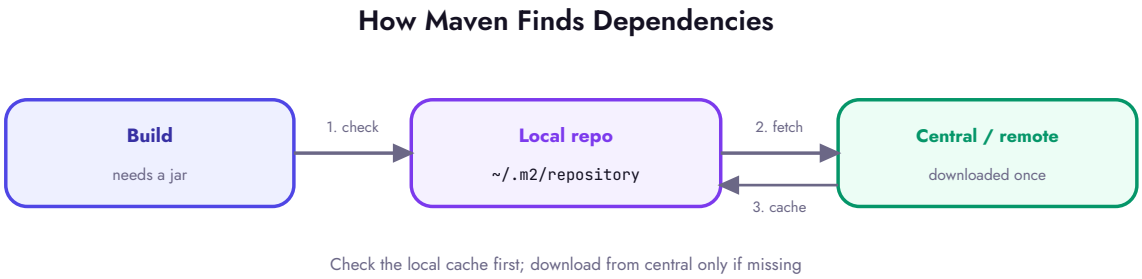
CHAPTER 13

Local, Central & Remote Repositories

Where Maven finds and stores artifacts — the local cache, Maven Central, and remote repos.

Maven does not keep libraries in your project — it fetches them from **repositories** and caches them. Understanding the three kinds — your local cache, the public Central repository, and team remote repositories — explains where dependencies come from and how resolution works.

How resolution flows



Check the local cache first; download from central only if missing

Maven checks the local cache first, then downloads from central or remote repositories.

Repository	Role
Local	your cache at ~/.m2/repository
Central	the huge public default repo
Remote	team / third-party repositories

The local repository

Every downloaded artifact is cached in `~/.m2/repository`. Maven checks here first, so a library is downloaded only once per machine, and offline builds work for anything already cached.

Maven Central

Maven Central is the default public repository — an enormous archive of open-source artifacts. Most dependencies resolve from it with no configuration at all.

Configuring remote repositories

```xml
<repositories>
  <repository>
    <id>company-releases</id>
    <url>https://nexus.bytemart.com/releases</url>
  </repository>
</repositories>
```

- Local first — no network needed for cached artifacts.
- Central — the default source for open-source libraries.
- Remote — for private or third-party artifacts.
- A cleared local cache forces a fresh re-download.

Cache first, then download

Maven resolves artifacts from your local cache first, then Maven Central, then any configured remote repositories — caching each download. This layered model means fast, mostly-offline builds and a single public source for open-source libraries.

MODULE 4 · REPOSITORIES

CHAPTER 14

Repository Managers

Why teams run a repository manager like Nexus or Artifactory, and what it provides.

Teams rarely let every build reach out to the public internet directly. A **repository manager** — Nexus or Artifactory — sits between your builds and the outside world, proxying and caching public artifacts and hosting your own. It is standard infrastructure for professional Java teams.

What a repository manager does

- **Proxies and caches** Maven Central, so downloads are fast and reliable.
- **Hosts private artifacts** your team publishes internally.
- Provides a **single URL** for all dependencies, public and private.
- Enables **governance** — security scanning, license and access policies.

Why proxy the public repository

Benefit	Why it matters
Speed	cached locally on the network
Reliability	builds work if Central is down
Control	vet and block risky artifacts
Auditability	know exactly what is used

Using it as a mirror

Point Maven at the manager as a **mirror** in `settings.xml`, so all requests flow through it — giving you one controlled gateway for every dependency.

xml

```xml
<!-- ~/.m2/settings.xml -->
<mirror>
  <id>company</id>
  <mirrorOf>*</mirrorOf>
  <url>https://nexus.bytemart.com/repository/maven-public</url>
</mirror>
```

A gateway for artifacts

A repository manager proxies and caches public artifacts, hosts private ones, and gives the team a single, governed source for every dependency. It makes builds faster and more reliable, and is the backbone of any serious team's Maven setup.

© 2026 ByteMart Digital

MODULE 4 · REPOSITORIES

CHAPTER 15

Publishing Artifacts

Sharing your own artifacts with the team by deploying them to a remote repository.

Building an artifact is only useful if others can consume it. `mvn deploy` publishes your artifact to a shared remote repository, where other projects and teams can depend on it — turning your library into a reusable, versioned building block.

install vs deploy, revisited

- `mvn install` — publishes to your **local** repository only.
- `mvn deploy` — publishes to a **remote** repository for everyone.
- Deploy is what makes an artifact available across the team.
- It runs the full build first — compile, test, package — then uploads.

Configuring where to deploy

`<distributionManagement>` tells Maven where to publish releases and snapshots. Snapshots go to a mutable snapshot repository; releases to an immutable release repository.

xml

```xml
<distributionManagement>
  <repository>
    <id>company-releases</id>
    <url>https://nexus.bytemart.com/releases</url>
  </repository>
  <snapshotRepository>
    <id>company-snapshots</id>
    <url>https://nexus.bytemart.com/snapshots</url>
  </snapshotRepository>
</distributionManagement>
```

Credentials and security

WATCH OUT

Deploy credentials belong in `~/.m2/settings.xml` (referenced by repository id), never in the POM — the POM is committed to source control. In CI, inject them as secrets.

Releases vs snapshots

- **Snapshots** deploy repeatedly during development; consumers get updates.
- **Releases** deploy once and are immutable — a fixed, reproducible version.
- Never overwrite a released version; publish a new one instead.
- A release process typically strips `-SNAPSHOT` and tags the version.

© 2026 ByteMart Digital

Deploy to share with the team

`mvn deploy` publishes your artifact to a shared remote repository so others can depend on it. Configure the target in `distributionManagement`, keep credentials in settings, and treat releases as immutable and snapshots as evolving.

MODULE 5 · PLUGINS & GOALS

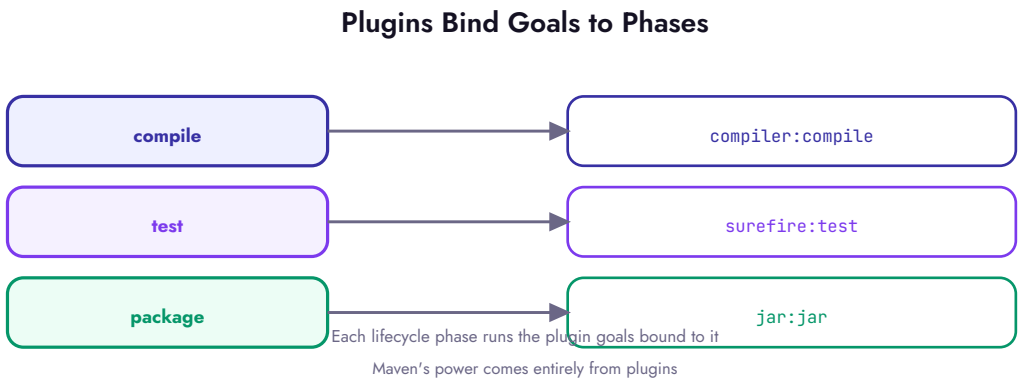
CHAPTER 16

How Plugins Work

The insight that Maven itself does almost nothing — plugins do all the real work.

Here is the key realisation about Maven: the core does very little. **Everything** — compiling, testing, packaging — is done by **plugins**, which provide **goals** bound to lifecycle phases. Understand this and Maven stops being a black box and becomes a coordinator of plugins.

Plugins provide goals



Each lifecycle phase runs the plugin goals bound to it; plugins do the real work.

A **plugin** is a bundle of related **goals**. The compiler plugin provides `compiler:compile`; the surefire plugin provides `surefire:test`. Phases are just slots these goals bind to.

Plugin coordinates and goals

```bash
# run a goal directly: plugin:goal
mvn compiler:compile
mvn surefire:test
mvn dependency:tree
mvn help:describe -Dplugin=compiler   # inspect a plugin's goals
```

The built-in bindings

The default lifecycle already binds core plugin goals to phases for you — which is why `mvn package` compiles, tests, and jars without any plugin configuration. You only configure plugins to change or extend that behaviour.

Phase	Default goal (plugin)
compile	compiler:compile
test	surefire:test
package	jar:jar
install	install:install

Two kinds of plugins

- **Build plugins** — run during the build (compiler, surefire, jar).
- **Reporting plugins** — generate reports for the site lifecycle.
- Community and vendor plugins cover almost any build task.
- You add plugins in the `<build><plugins>` section.

Maven coordinates plugins

Maven's core is a coordinator; plugins and their goals do all the actual work, bound to lifecycle phases. The default bindings give you a working build for free — configuring plugins is how you customise and extend it.

© 2026 ByteMart Digital

MODULE 5 · PLUGINS & GOALS

CHAPTER 17

Core Plugins

The essential plugins behind every build — compiler, surefire, jar, and their companions.

A handful of core plugins power the everyday build. You use them constantly, usually through the default bindings, but knowing what each does — and how to configure it — lets you take control when the defaults are not enough.

The essential plugins

Plugin	Provides
compiler	compiles Java sources
surefire	runs unit tests
failsafe	runs integration tests
jar / war	packages the artifact
resources	copies and filters resources
clean	deletes the target directory

Configuring the compiler

The compiler plugin controls the Java version and compilation options. Setting the release version is one of the most common plugin configurations.

xml

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <configuration>
    <release>21</release>
  </configuration>
</plugin>
<!-- or simply set the property: maven.compiler.release=21 -->
```

Surefire and failsafe

- **Surefire** runs **unit** tests in the `test` phase (`*Test.java`).
- **Failsafe** runs **integration** tests in `verify` (`*IT.java`).
- Failsafe separates slow integration tests from fast unit tests.
- Both produce reports under `target`.

The packaging plugins

© 2026 ByteMart Digital

The **jar** plugin (or **war** for web apps) assembles compiled classes and resources into the final artifact during the `package` phase, following your project's packaging type.

The plugins behind every build

Compiler, surefire, failsafe, and the packaging plugins do the core work of every Maven build. They run via default bindings, but you configure them — setting the Java release, separating test types — whenever the defaults need adjusting.

MODULE 5 · PLUGINS & GOALS

CHAPTER 18

Binding Goals to Phases

Attaching extra plugin goals to lifecycle phases to run custom steps automatically.

The default bindings cover the basics, but real builds need more — generating code, checking style, building an executable JAR. You add these by **binding a plugin's goal to a phase**, so it runs automatically as part of the build, exactly where it belongs.

Adding an execution

A plugin `<execution>` binds a goal to a phase. When that phase runs, so does the goal — no extra command needed.

xml

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-antrun-plugin</artifactId>
  <executions>
    <execution>
      <id>print-banner</id>
      <phase>compile</phase>       <!-- when to run -->
      <goals><goal>run</goal></goals>  <!-- what to run -->
    </execution>
  </executions>
</plugin>
```

A real example: building an executable JAR

The Shade plugin binds its `shade` goal to the `package` phase, so packaging produces a fat JAR with all dependencies included — automatically, every build.

xml

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-shade-plugin</artifactId>
  <executions>
    <execution>
      <phase>package</phase>
      <goals><goal>shade</goal></goals>
    </execution>
  </executions>
</plugin>
```

Why bind to phases

- The step runs **automatically** as part of the normal build.
- It runs at the **right point** — code generation before compile, packaging after.
- No one has to remember to run an extra command.

© 2026 ByteMart Digital

- CI gets the same behaviour as local builds, for free.

Wire extra steps into the build

Binding a plugin goal to a lifecycle phase with an execution makes custom steps — code generation, fat-JAR packaging, quality checks — run automatically at the right moment. This is how you extend the standard build to fit your project.

MODULE 5 · PLUGINS & GOALS

CHAPTER 19

Configuring Plugins

Passing configuration to plugins, and managing plugin versions across a project.

Plugins are configurable — the compiler's Java version, surefire's test patterns, the JAR's manifest. Passing configuration correctly, and pinning plugin versions, keeps builds predictable and reproducible across every machine and over time.

Plugin configuration

The `<configuration>` block passes settings to a plugin. It can apply to the whole plugin or to a specific execution.

xml

```
<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <configuration>
    <includes><include>**/*Test.java</include></includes>
    <parallel>classes</parallel>
    <threadCount>4</threadCount>
  </configuration>
</plugin>
```

Always pin plugin versions

WATCH OUT

Never leave plugin versions unspecified. Without a version, the build can pick up a different plugin release over time, breaking reproducibility. Pin every plugin version — ideally in `pluginManagement`.

pluginManagement

Like `dependencyManagement` for dependencies, `<pluginManagement>` centralises plugin versions and configuration — usually in a parent POM — so every module uses the same, pinned setup.

xml

```
<build>
  <pluginManagement>
    <plugins>
      <plugin>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.13.0</version>
      </plugin>
    </plugins>
  </pluginManagement>
</build>
```

© 2026 ByteMart Digital

Properties keep it tidy

xml

```
<properties>
  <java.version>21</java.version>
  <spring.version>6.1.0</spring.version>
</properties>
<!-- reference as ${java.version} throughout the POM -->
```

Configure and pin for reproducibility

Plugin configuration blocks pass settings; `pluginManagement` and pinned versions keep those settings consistent and reproducible. Always specify plugin versions — an unpinned plugin is a build that can change under you without warning.

MODULE 6 · MULTI-MODULE PROJECTS

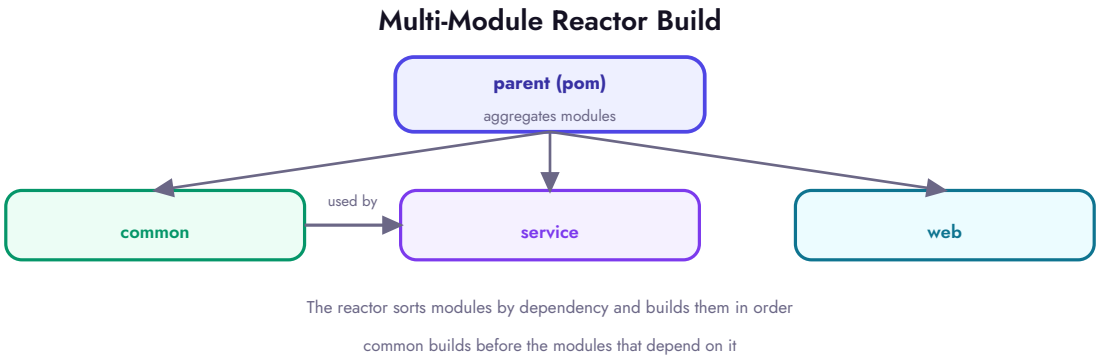
CHAPTER 20

Parent POMs & Inheritance

Sharing configuration across modules through POM inheritance from a common parent.

As projects grow into several modules, repeating dependency versions and plugin config in each becomes unmaintainable. A **parent POM** holds shared configuration, and child modules **inherit** it — defining common settings once, in one place.

The reactor at a glance



A parent POM aggregates modules; the reactor builds them in dependency order.

A parent POM

The parent has `<packaging>pom</packaging>` and holds shared configuration — properties, managed versions, plugin management — that children inherit.

xml

```
<project>
  <groupId>com.bytemart</groupId>
  <artifactId>shop-parent</artifactId>
  <version>1.0.0</version>
  <packaging>pom</packaging>

  <properties>
    <java.version>21</java.version>
  </properties>
  <dependencyManagement>...</dependencyManagement>
</project>
```

Inheriting in a child

```xml
<project>
  <parent>
    <groupId>com.bytemart</groupId>
    <artifactId>shop-parent</artifactId>
    <version>1.0.0</version>
  </parent>
  <artifactId>order-service</artifactId>   <!-- inherits group, version -->
</project>
```

What children inherit

- Properties, `dependencyManagement`, and `pluginManagement`.
- Common dependencies declared in the parent.
- The group and version, unless overridden.
- Repository and distribution configuration.

Define shared config once

A parent POM (packaging `pom`) holds properties, managed versions, and plugin config that child modules inherit. Inheritance eliminates duplication across modules — change a version in the parent, and every module follows.

MODULE 6 · MULTI-MODULE PROJECTS

CHAPTER 21

Aggregation & the Reactor

Building many modules together as one project, in the correct dependency order.

Inheritance shares configuration; **aggregation** builds modules together. An aggregator POM lists its modules, and Maven's **reactor** figures out the order to build them in — so a shared library is built before the services that depend on it.

Aggregation vs inheritance

These are separate concerns that often live in the same POM. **Inheritance** (via `<parent>`) shares config downward; **aggregation** (via `<modules>`) builds a set of modules together.

	Inheritance	Aggregation
Declared with		
Purpose	share config	build together
Direction	child -> parent	parent -> children

Listing modules

```xml
<project>
  <artifactId>shop-parent</artifactId>
  <packaging>pom</packaging>
  <modules>
    <module>shop-common</module>
    <module>order-service</module>
    <module>web</module>
  </modules>
</project>
```

The reactor

The **reactor** is the mechanism that builds a multi-module project. It reads the inter-module dependencies, sorts the modules topologically, and builds them in an order that respects those dependencies.

- Builds all listed modules with one command from the root.
- Sorts modules so dependencies build before dependents.
- `-pl` builds a subset; `-am` also builds what they need.
- A failure stops the reactor (or use `--fail-at-end`).

```bash
mvn install                 # build every module in reactor order
mvn -pl order-service -am install   # just this module + its deps
```

© 2026 ByteMart Digital

Build modules together, in order

Aggregation lists modules in a POM, and the reactor builds them together in dependency order — shared libraries before their consumers. Combined with inheritance, it lets one command build a whole multi-module project correctly.

MODULE 6 · MULTI-MODULE PROJECTS

CHAPTER 22

Structuring Multi-Module Projects

Organising modules and inter-module dependencies for a clean, maintainable build.

A well-structured multi-module project is easy to build, change, and reason about; a poorly-structured one fights you at every turn. A few principles — clear module boundaries, sensible inter-module dependencies, and shared configuration in the parent — keep it healthy as it grows.

Inter-module dependencies

One module depends on another by its coordinates, just like any dependency. The reactor ensures the dependency is built first.

xml

```xml
<!-- in order-service, depend on the common module -->
<dependency>
  <groupId>com.bytemart</groupId>
  <artifactId>shop-common</artifactId>
  <version>${project.version}</version>
</dependency>
```

A typical layout

- **parent** — pom packaging; shared config and module list.
- **common** — shared domain types and utilities.
- **service** modules — business logic, depend on common.
- **web / app** — the runnable module, depends on services.

Structuring principles

- Keep the dependency graph a **DAG** — no cycles between modules.
- Put shared versions and plugins in the **parent**.
- Use `${project.version}` so modules stay in lockstep.
- Split modules by **responsibility**, not to over-fragment.

WATCH OUT

Maven cannot build a project with a **cyclic** module dependency — module A needing B while B needs A. Keep the inter-module graph acyclic; a cycle is a sign the boundaries are wrong.

When to go multi-module

Multi-module shines when parts of a system are built and versioned together but kept separate for clarity. If modules are always released independently, separate repositories may serve better.

© 2026 ByteMart Digital

Clean modules, acyclic graph

A healthy multi-module project has clear responsibilities, an acyclic inter-module graph, shared config in the parent, and versions kept in lockstep. Structure it well and one command builds the whole thing; structure it poorly and it fights back.

© 2026 ByteMart Digital

MODULE 7 · PROFILES & PACKAGING

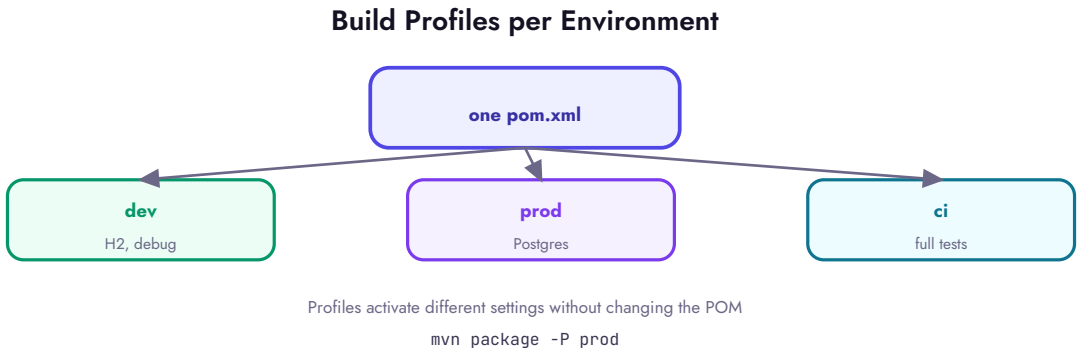
CHAPTER 23

Build Profiles

Varying the build for different environments and situations with activatable profiles.

The same project often needs to build differently in different situations — a lightweight local build, a full CI build, a production packaging. **Profiles** let you define these variations in one POM and activate the right one, without maintaining separate build files.

Profiles per environment



Profiles activate different settings for dev, prod, and CI from one POM.

Defining a profile

A `<profile>` overrides or adds properties, dependencies, or plugin configuration. It applies only when activated.

```xml
<profiles>
  <profile>
    <id>prod</id>
    <properties>
      <db.url>jdbc:postgresql://prod-db/shop</db.url>
    </properties>
  </profile>
</profiles>
```

Activating profiles

Activation	How
Explicit	-P prod on the command line
By property	when a property is set
By OS / JDK	matching the environment
By default	active unless another is chosen

```bash
mvn package -P prod          # activate the prod profile
mvn package -P ci,coverage   # activate several
```

Use profiles judiciously

WATCH OUT

Profiles that drastically change the build hurt reproducibility — a build that behaves differently depending on hidden activation is hard to reason about. Keep profiles focused, and prefer explicit activation over subtle automatic rules.

One POM, many build variations

Profiles let a single POM adapt to different environments and situations, activated explicitly or by conditions. Use them for genuine, focused variations — and keep activation obvious so builds stay predictable and reproducible.

MODULE 7 · PROFILES & PACKAGING

CHAPTER 24

Properties & Resource Filtering

Defining reusable values and injecting them into resource files at build time.

Properties keep your POM DRY by defining values once and referencing them everywhere. **Resource filtering** takes this further, substituting property values into resource files during the build — useful for stamping in versions and environment settings.

Properties

Define properties in the POM and reference them with `${...}`. Version numbers and shared settings are the classic use — change them in one place.

xml

```xml
<properties>
  <java.version>21</java.version>
  <jackson.version>2.17.0</jackson.version>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

Built-in properties

- `${project.version}`, `${project.artifactId}` — project coordinates.
- `${project.basedir}`, `${project.build.directory}` — paths.
- `${env.VAR}` — environment variables.
- Any property you define yourself.

Resource filtering

With filtering enabled, Maven replaces `${...}` placeholders in resource files with property values when copying them to `target` — baking build-time values into config files.

xml

```xml
<build>
  <resources>
    <resource>
      <directory>src/main/resources</directory>
      <filtering>true</filtering>
    </resource>
  </resources>
</build>
```

© 2026 ByteMart Digital

text

```
# application.properties  (before filtering)
app.version=${project.version}
# after the build -> app.version=1.4.0
```

WATCH OUT

Spring Boot uses `@...@` as its filtering delimiter, not `${...}`, to avoid clashing with Spring's own placeholders. If you use the Spring Boot parent, filter with `@project.version@`.

Define once, inject at build time

Properties centralise reusable values in the POM, and resource filtering substitutes them into resource files during the build. Together they keep configuration DRY and let you stamp versions and settings into your artifacts automatically.

© 2026 ByteMart Digital

MODULE 7 · PROFILES & PACKAGING

CHAPTER 25

Packaging Types & Assembly

Choosing the right packaging and assembling custom distributions of your project.

The **packaging** type tells Maven what kind of artifact to build — a JAR, a WAR, or a POM. When you need something beyond the standard artifact — a zip with scripts and config, or a distribution bundle — the **Assembly plugin** builds it.

Packaging types

Packaging	Produces
jar	a standard Java library (default)
war	a web application archive
pom	an aggregator / parent (no artifact)
ear	an enterprise application archive

xml

```xml
<packaging>jar</packaging>   <!-- the default; also war, pom, ear -->
```

The Assembly plugin

The **Assembly plugin** builds custom archives from a descriptor — bundling your JAR with dependencies, scripts, and configuration into a distributable zip or tarball.

xml

```xml
<plugin>
  <artifactId>maven-assembly-plugin</artifactId>
  <configuration>
    <descriptorRefs>
      <descriptorRef>jar-with-dependencies</descriptorRef>
    </descriptorRefs>
  </configuration>
</plugin>
```

What assembly can build

- A single JAR containing all dependencies (a simple fat JAR).
- A distribution zip with `bin`, `lib`, and `conf` folders.
- Custom bundles described by an assembly descriptor.
- Multiple output formats (zip, tar.gz) from one build.

© 2026 ByteMart Digital

TIP

For a simple runnable JAR, the Shade or Spring Boot plugin (next chapter) is usually easier than Assembly. Reach for Assembly when you need a **custom distribution layout** beyond a single JAR.

Package the right shape

Packaging chooses the artifact type — jar, war, or pom — and the Assembly plugin builds custom distributions with dependencies, scripts, and config bundled in. Pick the packaging for your project, and assemble when you need more than the standard artifact.

MODULE 7 · PROFILES & PACKAGING

CHAPTER 26

Executable & Fat JARs

Building a single, self-contained runnable JAR that includes all dependencies.

A plain JAR contains only your classes — to run it you still need every dependency on the classpath. An **executable fat JAR** bundles your code and all its dependencies into one file that runs with a single command, the standard way to ship a modern Java application.

The problem with a plain JAR

A standard JAR has no dependencies inside it and no defined entry point. Running it requires assembling the full classpath by hand — impractical for deployment.

The Shade plugin

The **Shade plugin** merges your classes and all dependencies into one JAR, and sets the main class — producing a self-contained, runnable artifact.

xml

```xml
<plugin>
  <artifactId>maven-shade-plugin</artifactId>
  <executions>
    <execution>
      <phase>package</phase>
      <goals><goal>shade</goal></goals>
      <configuration>
        <transformers>
          <transformer implementation=
            "...ManifestResourceTransformer">
            <mainClass>com.bytemart.App</mainClass>
          </transformer>
        </transformers>
      </configuration>
    </execution>
  </executions>
</plugin>
```

The Spring Boot plugin

For Spring Boot apps, the Spring Boot Maven plugin builds an executable, layered JAR with a smart launcher — no Shade configuration needed.

bash

```bash
mvn package            # produces target/app-0.0.1.jar
java -jar target/app-0.0.1.jar   # runs, dependencies included
```

Choosing an approach

- **Spring Boot plugin** — for Spring Boot apps; layered and optimised.
- **Shade plugin** — general-purpose fat JARs for any app.
- **Assembly plugin** — when you need a full distribution, not just a JAR.
- Watch for resource merge conflicts when many JARs are combined.

One JAR that just runs

An executable fat JAR bundles your code and every dependency into a single runnable file — built by the Shade plugin for general apps or the Spring Boot plugin for Boot apps. It is the standard, deployment-friendly way to ship a Java application.

© 2026 ByteMart Digital

MODULE 8 · QUALITY & PRACTICE

CHAPTER 27

Testing with Maven

Running unit and integration tests as part of the build, and reporting on them.

Testing is built into Maven's lifecycle. Unit tests run in the `test` phase and integration tests in `verify`, so a normal build already runs your tests. Knowing how the two test plugins differ — and how to separate fast from slow tests — keeps the build both safe and quick.

Unit tests with Surefire

The Surefire plugin runs unit tests during the `test` phase. By convention it runs classes named `*Test`, and a failing test fails the build.

```bash
mvn test              # compile and run unit tests
mvn package           # unit tests run before packaging
```

Integration tests with Failsafe

The Failsafe plugin runs **integration** tests (named `*IT`) in the `verify` phase — after packaging. Separating them keeps the fast unit-test feedback loop quick.

	Surefire	Failsafe
Runs	unit tests	integration tests
Phase	test	verify
Naming	*Test	*IT

Why the separation matters

- Unit tests are fast — run them on every build for quick feedback.
- Integration tests are slower — run them in `verify`, often in CI.
- Failsafe ensures cleanup runs even if an integration test fails.
- You can skip integration tests locally but never in the release pipeline.

Coverage and reports

Plugins like **JaCoCo** bind to the build to measure test coverage and can fail the build below a threshold — turning coverage into an enforced quality gate.

> **Tests are part of the build**
> Surefire runs unit tests in `test` and Failsafe runs integration tests in `verify`, so the standard lifecycle already tests your code. Separate fast from slow tests, add coverage gates, and a passing build genuinely means working software.

© 2026 ByteMart Digital

MODULE 8 · QUALITY & PRACTICE

CHAPTER 28

Maven in CI/CD

Running Maven builds reliably in continuous integration and delivery pipelines.

Maven and CI/CD fit together naturally: a pipeline runs the same Maven commands you run locally, on every commit. A few practices — clean builds, dependency caching, and reproducible configuration — make those pipeline builds fast, reliable, and trustworthy.

The pipeline build

A CI build typically runs a clean, full build with tests, then — on the right branch — deploys the artifact. It uses the wrapper so the Maven version is pinned.

```bash
./mvnw clean verify      # clean build + unit and integration tests
./mvnw deploy            # publish the artifact (on release branches)
```

Practices for reliable CI

- Always **clean** in CI so no stale output survives.
- Use the **wrapper** to pin the Maven version.
- **Cache** the local repository between runs for speed.
- Run through a **repository manager** for fast, reliable downloads.
- Fail the build on test failures — never skip tests in CI.

Caching dependencies

Downloading all dependencies on every CI run is slow. Cache the `~/.m2/repository` directory between runs — every CI system supports this — so only new dependencies are fetched.

Reproducibility in CI

- Pin plugin and dependency versions — no surprises over time.
- Avoid SNAPSHOT dependencies in release builds.
- Keep credentials and secrets out of the POM; inject them in CI.
- Make the build deterministic: same input, same artifact.

Same build, automated

CI runs the same Maven commands you run locally, on every change — a clean `verify` that tests everything. Pin versions, cache the local repo, run tests, and keep builds reproducible, and your pipeline becomes a reliable gate to production.

© 2026 ByteMart Digital

MODULE 8 · QUALITY & PRACTICE

CHAPTER 29

Maven vs Gradle

How Maven compares to Gradle, and how to choose between the two build tools.

Maven's main alternative is **Gradle**. Both manage dependencies and builds, but they differ in philosophy: Maven is declarative XML with strong conventions, Gradle is a flexible, scriptable build. Understanding the trade-offs helps you choose — and work in either.

The core difference

	Maven	Gradle
Build file	XML (pom.xml)	Groovy/Kotlin script
Style	declarative	scriptable
Speed	good	faster (incremental, caching)
Flexibility	conventional	highly customisable

Maven's strengths

- **Convention** — a standard, predictable build everyone understands.
- **Simplicity** — declarative XML with little logic to reason about.
- **Stability** — mature, ubiquitous, deeply supported everywhere.
- Easy for newcomers and easy to read across projects.

Gradle's strengths

- **Performance** — incremental builds and a build cache.
- **Flexibility** — a real language for custom build logic.
- The default for **Android** and increasingly common on the server.
- Concise build scripts for complex builds.

Choosing

There is no universal winner. Maven suits teams that value convention, readability, and stability; Gradle suits those needing build performance and heavy customisation. Both are excellent — many organisations happily standardise on either.

> **Convention vs flexibility**
> Maven offers declarative, convention-driven builds that are simple and predictable; Gradle offers scriptable, high-performance builds with more flexibility. Choose Maven for convention and stability, Gradle for speed and customisation — both are solid choices.

© 2026 ByteMart Digital

MODULE 8 · QUALITY & PRACTICE

CHAPTER 30

Best Practices & Troubleshooting

The habits that keep Maven builds healthy, and how to diagnose the common problems.

Maven rewards discipline. A handful of good habits keep builds fast, reproducible, and easy to maintain — and a handful of diagnostic techniques resolve the problems that do arise. This closing chapter gathers both, drawn from the whole guide.

Best practices

- **Pin every version** — dependencies and plugins — for reproducibility.
- Centralise versions in **dependencyManagement** and a parent POM.
- Commit and use the **Maven Wrapper**.
- Keep dependencies minimal; run `dependency:analyze` regularly.
- Never commit **secrets** to the POM; use `settings.xml` or CI.
- Avoid SNAPSHOT dependencies in released builds.

Diagnosing common problems

Symptom	First check
wrong library version	mvn dependency:tree -Dverbose
unused / missing deps	mvn dependency:analyze
mystery config	mvn help:effective-pom
build behaves oddly	mvn -X (debug output)

The usual culprits

- **Dependency conflicts** — a mediated version breaking a library (check the tree).
- **Stale local cache** — a corrupted artifact; delete it and re-download.
- **Unpinned plugins** — a build that changed under you.
- **SNAPSHOT drift** — non-reproducible builds from mutable dependencies.

The guiding principle

Above all, keep builds **reproducible**: the same source and configuration should always produce the same artifact. Pinned versions, clean builds, and no hidden state are what make a Maven build something you can trust.

> **Discipline makes builds trustworthy**
> Healthy Maven builds come from pinned versions, centralised management, the wrapper, and minimal dependencies — and most problems yield to `dependency:tree`, `analyze`, and `-X`. Keep builds reproducible, and Maven becomes a dependable foundation.

© 2026 ByteMart Digital